

Module : M202 – Développement front-end
Filière : Développement digital – option web full stack

TP 1

Exercice 1 :

1. Soit le code suivant:

```
let numbers=[3,7,2,5,9,4,10]  
console.log(numbers.filter(n=>n>=5))
```

Ce code affiche :

- a. 5
- b. Erreur
- c. 7,5,9,10
- d. [7,5,9,10]

2. Soit le code suivant:

```
let numbers=[3,7,2,5,9,4,10]  
console.log(numbers.find(n=>n>=5))
```

Ce code affiche :

- a. 5
- b. 7
- c. [7,5,9,10]
- d. 7,5,9,10
- e. undefined

3. Soit

```
let numbers=[3,7,2,5,9,4,10]  
console.log(numbers.find(n=>n==6))
```

Ce code affiche :

- a. 5
- b. 7
- c. [7,5,9,10]
- d. Undefined

4. Soit

```
let numbers=[3,7,2,5,9,4,10]  
console.log(numbers.map(n=>n*2))
```

Ce code affiche :

- a. 6
- b. 6,14,4,10,18,8,20
- c. [6,14,4,10,18,8,20]
- d. erreur

5. Soit

```
let numbers=[3,7,2,5,9,4,10]
console.log(numbers.reduce(function(total,n){return total+n},0))
```

Ce code affiche :

- a. 30
- b. 40
- c. Erreur
- d. undefined

6. Soit le code suivant :

```
function salutation(nom, presentation){ presentation(nom) }
function hello(nom){ console.log(`hello ${nom.toUpperCase()}`) }
function salut(nom){ console.log(`salut ${nom.toUpperCase()}`) }
salutation("rami",hello)
salutation("fahmi",salut)
```

Ce code affiche :

- a. Rami hello fahmi salut
- b. hello RAMI salut FAHMI
- c. erreur
- d. undefined

7. Soit le code suivant :

```
let cmpt={second:0}
const incremente=()=>{
  cmpt.second++;
  if(cmpt.second==60) cmpt.second=0;
  console.log(cmpt.second)
}
const compteur=setInterval(incremente,1000)
```

- a. affiche des valeurs de 1 à 1000
- b. affiche des valeurs de 1 à 59 puis recommence de 0 à 59 plusieurs fois
- c. erreur
- d. undefined

Exercice 2 : Manipulation des classes

Manipulation des classes et objets en javascript ES6

- a) Créer la classe Document contenant les attributs id et dateEdition
- b) Créer la classe Livre qui hérite de la classe Document contenant en plus les attributs titre et auteur.
- c) Créer deux instances de Document
- d) Créer trois instances de Livre
- e) Créer la méthode infoLivre qui retourne les informations du livre exemple :

```
Livre: id:10 titre:POO Auteur: Rami date Edition:10/05/2020
```

Exercice 3 : Manipulation des Array

Soit le tableau suivant : `const input = [1, 2, 3, 4, 5]`

Utiliser la méthode `map` pour créer un tableau contenant le carré des éléments de `nums`

Résultat : `[1, 4, 9, 16, 25]`

Exercice 4 : Manipulation des Array

Soit le tableau suivant : `const input = [1, -2, 3, 4, -5]`

Utiliser la méthode `filter` pour créer un tableau contenant les éléments positifs de `nums`

Résultat : `[1, 3, 4]`

Utiliser la méthode `reduce` pour calculer la somme des éléments de `nums`

Résultat : `-1`

Utiliser la méthode `filter` et `reduce` pour calculer la somme des éléments positifs de `nums`

Résultat : `8`

Exercice 5 : Manipulation des Array

Soit

```
let livres=[{id :10,titre : 'POO',auteur : 'RAMI',prix :300},  
{id :11,titre : 'JS ES6',auteur : 'FAMI',prix :230},  
{id :12,titre : 'Algorithme',auteur : 'KARIMI',prix :330},  
{id :13,titre : 'HTMH& CSS',auteur : 'RAMI',prix :340}]
```

1. Créer la liste titres contenant tous les titres à partir de Livres utiliser `map`
2. Créer la liste titres contenant tous les livres de l'auteur 'RAMI' à partir de Livres `filter`
3. Chercher le livre dont le id égal à 12 utiliser `find`
4. Calculer le total des prix utiliser `reduce`
5. Créer un Array `mesLivres` copie de livre

Exercice 6 : Modules

Créez deux fichiers JavaScript distincts, l'un contenant une fonction `add` qui additionne deux nombres, et l'autre contenant une fonction `multiply` qui multiplie deux nombres. Importez ces fonctions dans un troisième fichier JavaScript et utilisez-les pour effectuer des opérations mathématiques.

Exercice 7 : Etude de cas

On souhaite développer une application de gestion des tâches.

Gestion des Tâches

- ☐ Tache 1
- ☐ Tache 2
- ☐ Tache X

1. Créer une structure HTML de base : Créez un fichier HTML avec les éléments nécessaires, tels qu'un champ de texte pour ajouter des tâches, un bouton "Ajouter", une liste pour afficher les tâches et un conteneur pour afficher les tâches.
2. Créez une classe Tache avec des propriétés telles que id, description et terminee. Cette classe sera utilisée pour représenter chaque tâche.
3. Initialiser une liste de tâches vide : Dans le fichier JavaScript, initialisez une liste vide (un tableau) pour stocker les tâches. Vous pouvez utiliser const pour déclarer cette liste.
4. Créer des fonctions pour gérer les tâches :
 - a. ajouterTache(description) : Créez une fonction pour ajouter une nouvelle tâche au tableau des tâches en utilisant la classe Tache.
 - b. supprimerTache(id) : Créez une fonction pour supprimer une tâche du tableau en utilisant findIndex() pour trouver l'index de la tâche à supprimer.
 - c. marquerTacheTerminee(id) : Créez une fonction pour marquer une tâche comme terminée ou non en utilisant find() pour trouver la tâche correspondante.
5. Créer une fonction renderTaches() : Créez une fonction pour afficher les tâches dans la liste en utilisant map() pour générer les éléments HTML nécessaires et les ajouter à la liste.
6. Ajouter des écouteurs d'événements : Ajoutez un écouteur d'événements au bouton "Ajouter" pour détecter le clic et appeler ajouterTache() lorsque le bouton est cliqué.
7. Gérer l'affichage initial : Appelez renderTaches() pour afficher les tâches initiales lorsque la page est chargée.
8. Organisez votre code en modules distincts pour gérer différentes parties de l'application